

Projet SMART

(Smart Merchandise Automated Recognition Technology)

Équipe

- Bastian FABRE
 - Samy MOSA
 - Julien VON DER MARCK
-

Objectif

Développer un programme en Python utilisant la Computer Vision pour la reconnaissance automatique d'un ensemble prédéfini de produits. La détection sera effectuée à partir d'une image, d'une vidéo ou du flux caméra d'un ordinateur en temps réel.

Présentation

Ce rapport de projet vise à détailler les expérimentations qui ont été conduites ainsi que les analyses qui en ont découlées. À travers ce document, vous comprendrez le cheminement qui nous a amenés à la meilleure version de notre model.

Le model a été entraîné sur la base des algorithmes de machine learning de détection d'objets YOLO en version 11 et taille nano. La plateforme de MLOps Picsellia a été utilisée afin d'annoter notre dataset, de garder en mémoire nos différentes expérimentations ainsi que leurs métriques et enfin stocker nos models.

Lien vers notre meilleur model final : [Picsellia](#)

Les premiers essais

Au tout début, nous avons essayé de faire 5 epochs. N'ayant pas vraiment d'expérience dans ce domaine, nous ne pouvions pas nous douter de ce qui allait arriver.

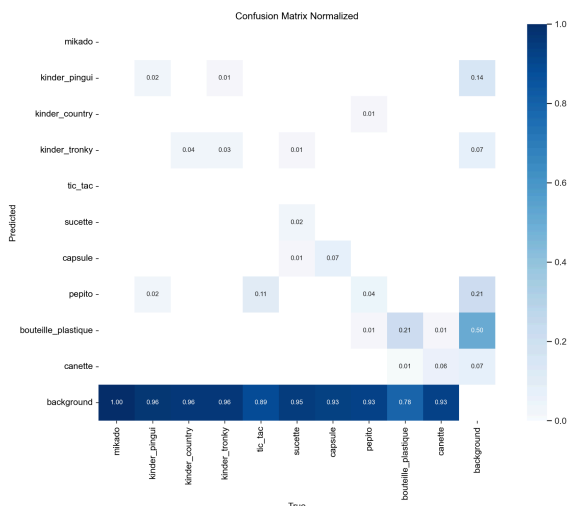


Figure 1 : Matrice de confusion (5 epochs)

Comme nous le montre cette figure, un training de **5 epochs** n'est clairement pas assez long. La plupart des objets ne sont pas identifiés et tombent donc dans le "background".

Il faut donc continuer.

Pour accélérer la progression, nous avons ensuite opté pour un **batch_size** plus grand en passant à **32** au lieu de 16 auparavant. Pour optimiser les performances, la taille des **images** a été réduite de 640 à **512**. Nous sommes maintenant à **10 epochs**, ce qui est donc le double.

Ici, l'évolution est encore minime mais on commence à avoir moins d'objets oubliés. La ligne background accueille des cases moins foncées, ce qui montre que les objets ont plus tendance à être trouvés. (Même si cela ne veut pas dire qu'ils sont bien identifiés.)

Il faut peut-être faire énormément d'epochs ?

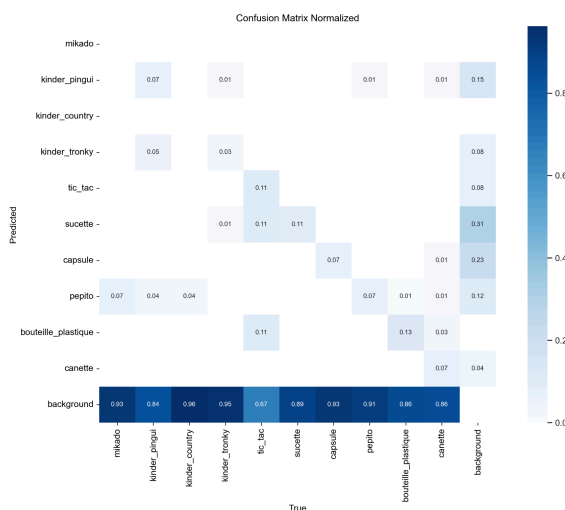


Figure 2 : Matrice de confusion (10 epochs)

Adieu Apple Silicon...

Il faut savoir que les entraînements ont été fait sur un MacBook Pro M1 Pro de 16 pouces. Bien que nous ne disposons pas de carte graphique NVIDIA, il est tout à fait possible d'entraîner notre model. Malheureusement, nous ne pouvons donc pas utiliser CUDA. Le backend MPS sur MacOS permet d'utiliser la puissance du GPU pour l'apprentissage automatique. Les résultats n'ont jamais été très concluants et nous n'avons pas osé faire beaucoup d'epochs car l'entraînement est long. En effet, même si l'apprentissage est possible, il n'est pas des plus optimisés et une carte NVIDIA permet de bien meilleures performances. Bastian a donc pris l'opportunité d'utiliser le PC portable consacré à son travail en entreprise pour continuer le projet. Nous passons maintenant sur un GPU NVIDIA RTX 3050 Ti avec CUDA.

Plus de puissance, ça change tout

Avec notre nouveau GPU en main, l'entraînement est devenu beaucoup plus rapide. Il fallait attendre plus de 5 minutes pour une epoch avec le M1 Pro, ce qui est relativement long. Désormais, moins de 15 secondes suffisent pour une epoch. L'écart de performance et de vitesse est vraiment fulgurant et nous aidera à bien avancer dans notre projet. Même si tout est plus rapide, nous avons opté pour un **batch_size** réduit de **8**.

1/5	0G	0.993	3.944	1.153	22	640: 100%	36/36 [05:08<00:00, 8.56s/it]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	7/7 [00:46<00:00, 6.57s/it]
all	193	585	0.00978	0.615	0.0697	0.0469	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
2/5	0G	1.096	3.611	1.209	68	640: 6%	2/36 [00:15<04:28, 7.89s/it]

Figure 3 : Logs d'entraînement sur M1 Pro

11/200	1.386	1.1	1.732	1.187	19	640: 100%	72/72 [00:13<00:00, 5.34it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	12/12 [00:02<00:00, 5.60it/s]
all	190	481	0.571	0.337	0.393	0.286	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
12/200	1.416	1.097	1.705	1.184	16	640: 100%	72/72 [00:13<00:00, 5.42it/s]
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	12/12 [00:02<00:00, 5.26it/s]
all	190	481	0.502	0.455	0.505	0.379	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
13/200	1.296	1.21	1.808	1.244	39	640: 8%	6/72 [00:01<00:14, 4.67it/s]

Figure 4 : Logs d'entraînement sur RTX 3050 Ti

Toujours laisser la fenêtre ouverte

Si vous l'aviez remarqué dans la figure précédente, ce gain de performance nous a permis d'oser un entraînement plus coriace. Nous avons franchi la barre des 200 epochs. Petit détail peu amusant : nous sommes sur un PC portable Windows. De ce fait, il nous a fallu du temps pour comprendre que la veille Windows empêchait l'apprentissage automatique de se faire correctement. Solution simple trouvée : ne jamais être en veille et toujours rester allumé pour garder les performances au maximum en restant branché.

Qui peut le plus peut le moins

Bref, les 200 epochs sont passés. Que dire ? Les courbes de training sont très intéressantes. Il y a bien suffisamment de données pour avoir de belles informations et mieux comprendre ce qui se trame. Il faut surtout retenir que les métriques de mAP sont très bonnes. La mAP (ou mean Average Precision) est une métrique qui évalue la précision et le rappel d'un modèle de détection d'objets, en faisant la moyenne des précisions moyennes pour chaque classe. On comprend donc que le model est très performant. Il n'y a pas de signe d'overfitting donc il faut encore continuer l'entraînement.

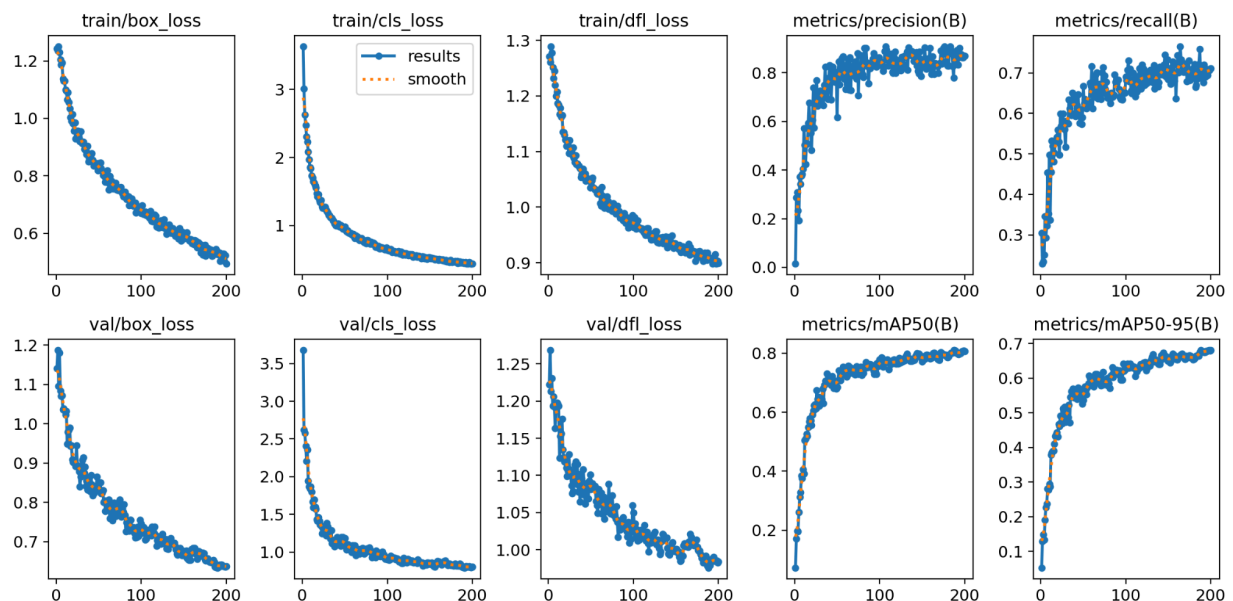


Figure 5 : Courbes de résultats après 200 epochs

Le batch_size passé à 8 et les epochs qui franchissent le palier des 200, la matrice de confusion est bien plus homogène. L'entraînement est de bon augure et cela présage du bon pour la suite.

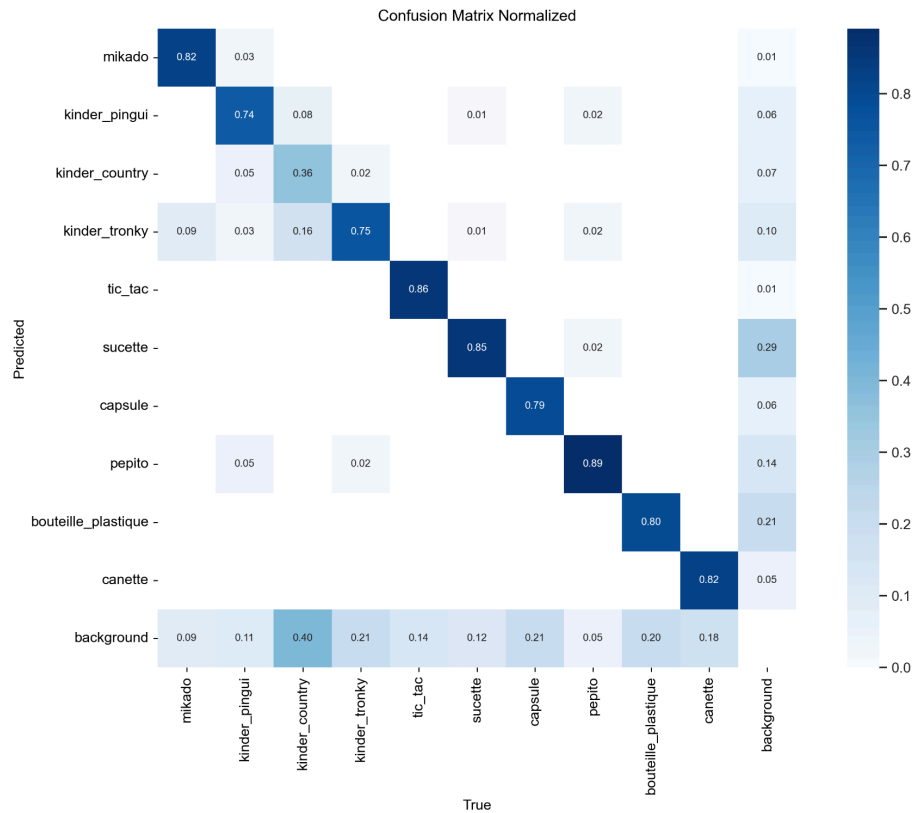


Figure 6 : Matrice de confusion après 200 epochs

Tous les voyants sont au vert, voici donc un nouveau training à 300 epochs. La légende à droite nous confirme que le modèle est toujours plus homogène. Il faut donc encore continuer les entraînements et viser toujours plus d'epochs.

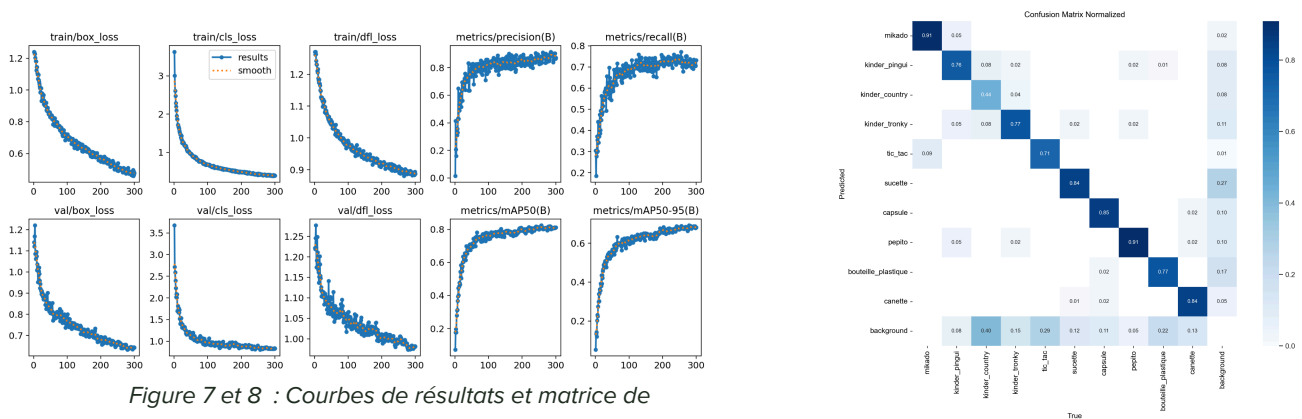


Figure 7 et 8 : Courbes de résultats et matrice de confusion après 300 epochs

Le début de la fin

Nous voici après 500 epochs d'apprentissage automatique pour la détection d'objets. Pour une fois, le training s'est arrêté avant la fin. La patience (ici à 100) s'est déclenchée. En d'autres termes, après 100 epochs de suite, le model n'a pas montré d'amélioration et s'est donc arrêté pour éviter un over-fitting.

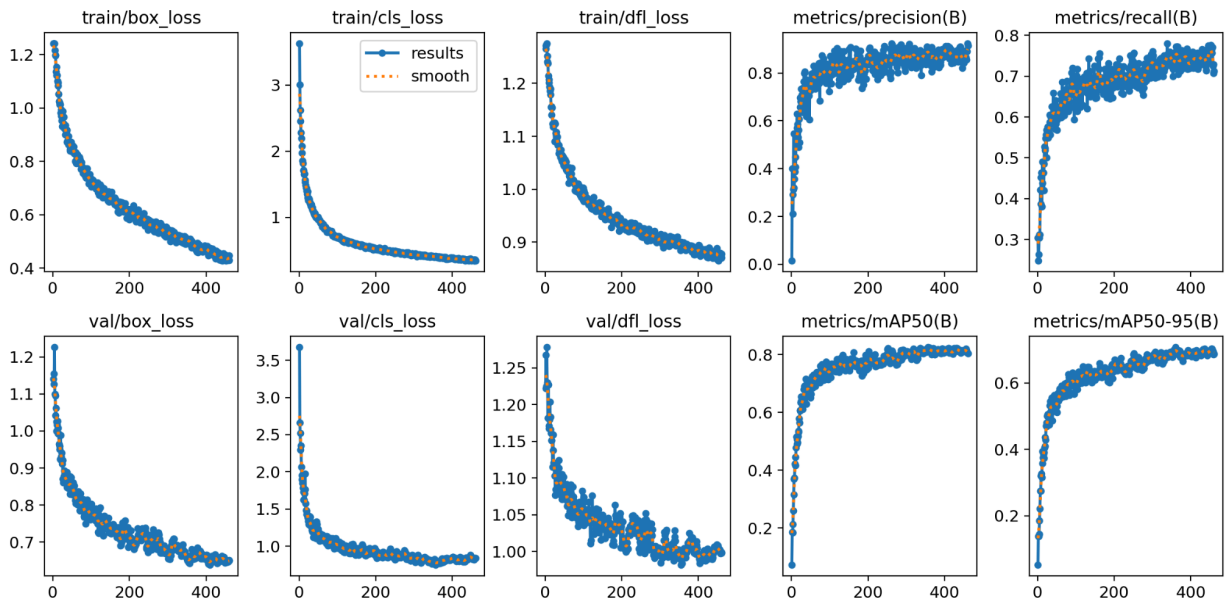


Figure 7 : Courbes de résultats après 500 epochs

Le terminal nous montre bien cette information. Le training s'est arrêté à l'epoch 461, car il n'y a pas eu d'améliorations après 100 epochs. La meilleure epoch est donc la 361.

```
EarlyStopping: Training stopped early as no improvement observed in last 100 epochs. Best results observed at epoch 361, best model saved as best.pt.
To update EarlyStopping(patience=100) pass a new patience value, i.e. 'patience=300' or use 'patience=0' to disable EarlyStopping.

461 epochs completed in 2.175 hours.
Optimizer stripped from results\train13\weights\last.pt, 5.5MB
Optimizer stripped from results\train13\weights\best.pt, 5.5MB

Validating results\train13\weights\best.pt...
Ultralytics 8.3.43 Python-3.11.9 torch-2.5.1+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4096MiB)
YOLO11n summary (fused): 238 layers, 2,584,102 parameters, 0 gradients, 6.3 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	190	481	0.873	0.747	0.821	0.68
mikado	11	11	0.848	0.818	0.882	0.862
kinder_pingui	35	38	0.803	0.763	0.85	0.733
kinder_country	20	25	0.84	0.52	0.591	0.474
kinder_tronky	42	53	0.88	0.736	0.794	0.644
tic_tac	7	7	0.994	0.857	0.891	0.601
sucette	52	81	0.897	0.741	0.854	0.678
capsule	43	61	0.917	0.725	0.84	0.687
pepito	45	55	0.832	0.873	0.904	0.723
bouteille_plastique	53	88	0.84	0.659	0.781	0.65
canette	50	62	0.881	0.774	0.824	0.751

```
Speed: 0.2ms preprocess, 12.8ms inference, 0.0ms loss, 1.2ms postprocess per image
Results saved to results\train13
Entraînement YOLOv11 lancé avec succès.
```

Figure 8 : Terminal à la fin du training montrant l'arrêt à l'epoch 461

Expérimentations extrêmes

Après plusieurs bidouillages d'hyperparamètres et d'analyses de résultats, nous ne parvenons pas à avoir un meilleur model. Il ne reste plus que quelques solutions : abandonner et rester avec ce model ou persévérer et opter pour plus d'expérimentations avec de plus gros changements.

En tentant le tout pour le tout, nous décidâmes donc d'essayer un **batch_size** de **64**. Quelle erreur. Premièrement, l'entraînement était très long... Plus de 15 heures en tout, pour aucune amélioration, voire même une baisse de performances pour notre model.

```
500 epochs completed in 15.826 hours.
```

Figure 9 : Terminal affichant la fin du training de 500 epochs avec batch_size de 64

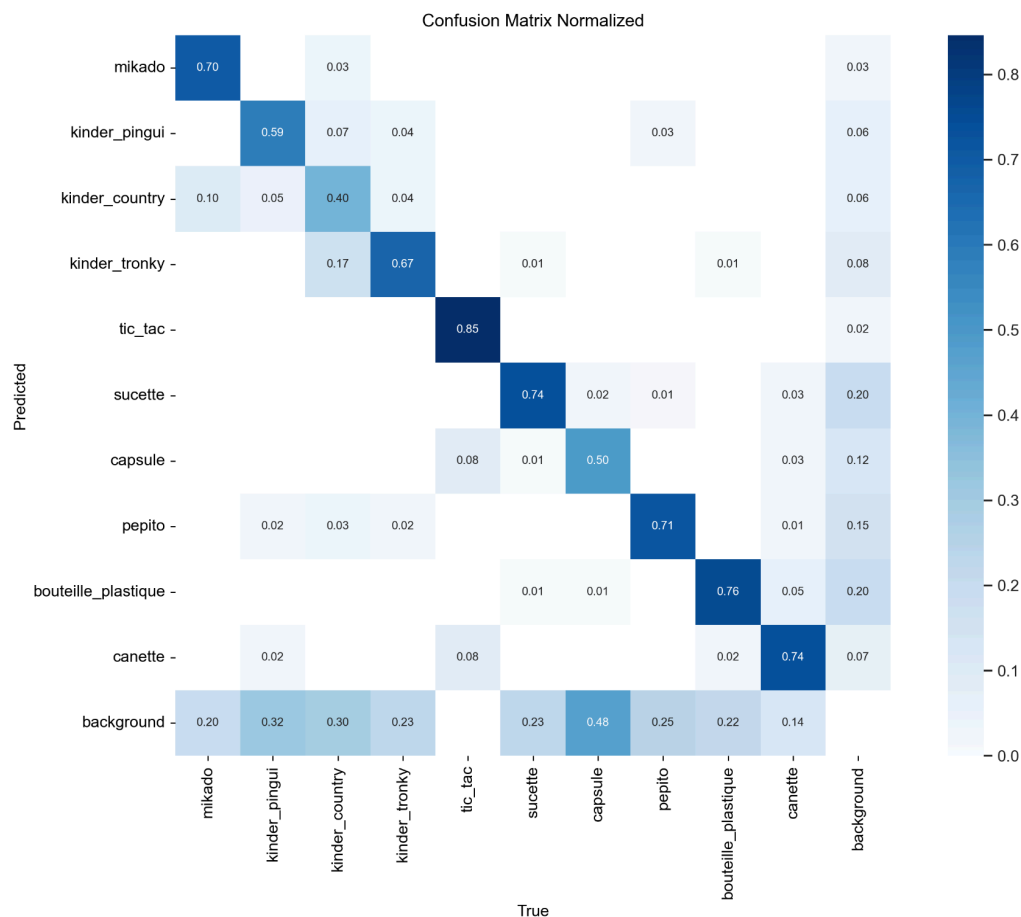


Figure 10 : Matrice de confusion à la fin du training de 500 epochs avec batch_size de 64

[illegible]

Google Collab permet l'utilisation gratuite de cartes NVIDIA Tesla T4. L'entraînement était donc très rapide (8 secondes maximum par epoch). Néanmoins, l'environnement est assez limité. L'espace de stockage reste temporaire et le temps d'exécution maximale n'est pas assez long pour pouvoir finir notre entraînement. Cette expérience fut intéressante mais elle n'a pas pu se finir.

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
130/500	1.32G	0.4651	0.4629	0.9047	9	640:	100% 76/76 [00:14<00:00, 5.38it/s]
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 13/13 [00:02<00:00, 5.79it/s]
	all	201	538	0.854	0.526	0.664	0.542
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
131/500	1.3G	0.4639	0.4612	0.8956	6	640:	100% 76/76 [00:12<00:00, 5.91it/s]
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 13/13 [00:02<00:00, 6.25it/s]
	all	201	538	0.719	0.574	0.63	0.514
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
132/500	1.38G	0.4672	0.4581	0.8981	18	640:	100% 76/76 [00:12<00:00, 6.12it/s]
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 13/13 [00:01<00:00, 6.63it/s]
	all	201	538	0.735	0.572	0.641	0.523
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
133/500	1.33G	0.4567	0.4495	0.8941	8	640:	100% 76/76 [00:12<00:00, 6.09it/s]
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 13/13 [00:01<00:00, 6.73it/s]
	all	201	538	0.816	0.56	0.663	0.532
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
134/500	1.31G	0.4933	0.4829	0.9094	24	640:	99% 75/76 [00:13<00:00, 6.71it/s]

7

Le tout pour le tout... ou rien

Après plusieurs recherches, nous sommes tombés sur vast.ai, une plateforme de location de GPU dans le cloud. Nous avons donc pu louer une configuration avec 4 GPU NVIDIA RTX 4090 pour entraîner notre model. Est-ce que ces efforts se sont révélés être payants ? Oui, car le coût est de 1.084\$ par heure.

4x RTX 4090		H12SSL-i	↑585.5 Mbps ↓1825.1 Mbps	
327.2 TFLOPS	1.8/96.0 GB	PCIe 4.0, 16x	23.5 GB/s	
Max CUDA: 12.8	900.2 GB/s	AMD EPYC 7R32 ...	KINGSTON SFYR...	
		96.0/96 cpu	0/258 GB	3442 MB/s
				13.0/32.0 GB

Figure 12 : Spécifications de l'instance loué sur vast.ai

Autrement... Pas vraiment. Cette grande configuration est parfaite pour tester plus de grands paramètres et refaire des entraînements avec un **batch_size** à **32** et **16** mais le model n'a jamais été mieux. Il est donc vraiment inutile (dans notre situation) d'avoir un grand batch_size. Au moins nous avons essayé.

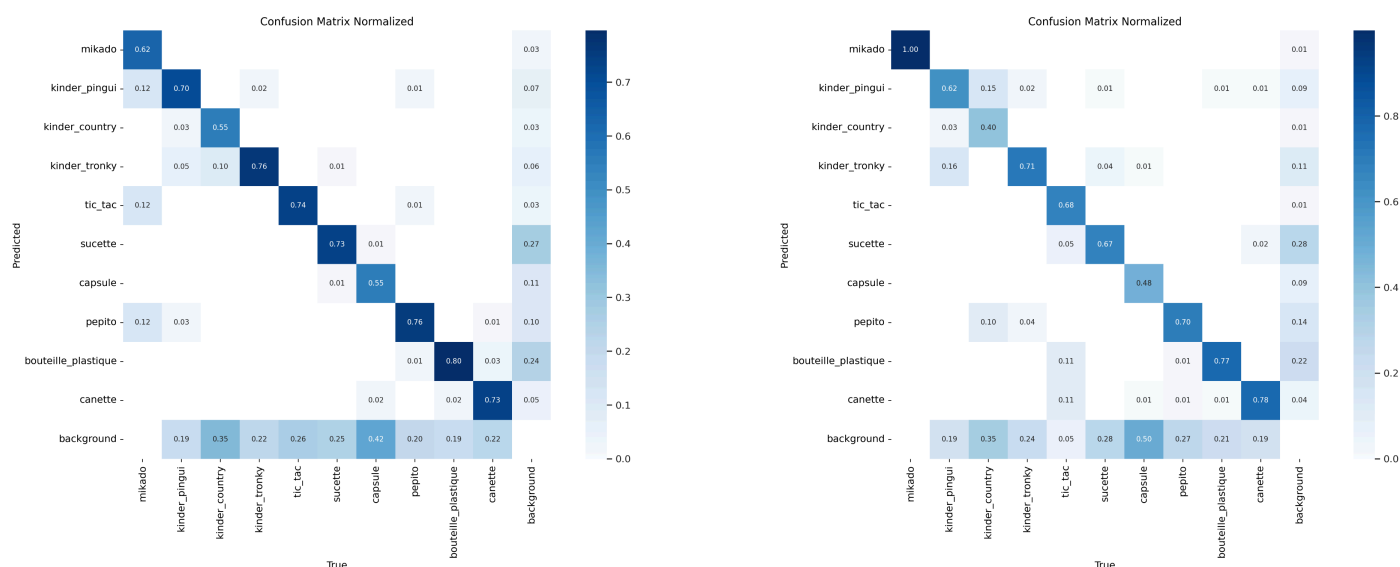


Figure 12 et 13 : Matrice de confusion avec batch_size à 32 et 16

Nous avons donc conclu que nous avons fait le meilleur possible à la première tentative des 500 epochs. Malgré des changements de hyperparamètres, le model n'a jamais été plus performant.

Notre meilleur model

Tout au long du projet, de nombreuses expérimentations ont été faites. La pipeline de training évoluant peu à peu, le système de logs n'a pu être prêt qu'à partir d'un certain moment. Nous ne pouvons malheureusement pas fournir la petite liste d'hyperparamètres qui était dans notre script pour notre meilleur model. Cependant, même si les métriques et autres informations n'ont pas été envoyées sur Picsellia, il reste le dossier de train. Voici donc différentes métriques et la liste complète des hyperparamètres qui attestent des résultats. Les plus gros ajustements sont la **batch_size** à **8** et la **learning rate** à **0.001** (qui ont baissé peu à peu). Finalement, ce ne sont donc pas des gros paramètres qui ont créé ce meilleur model. Inutile d'avoir des grandes images et un grand batch_size. Le plus important est l'optimisation.

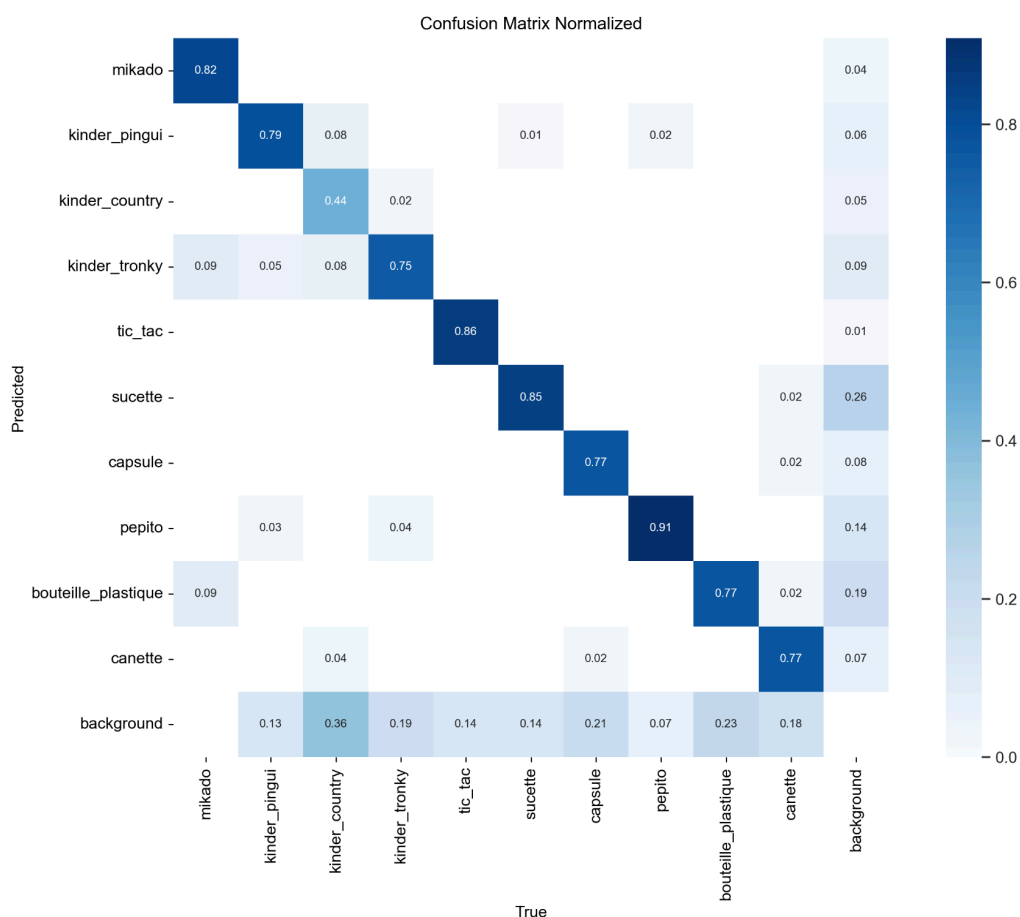


Figure 14 : Matrice de confusion du meilleur model actuel

Lien vers ce meilleur model : [Picsellia](#)

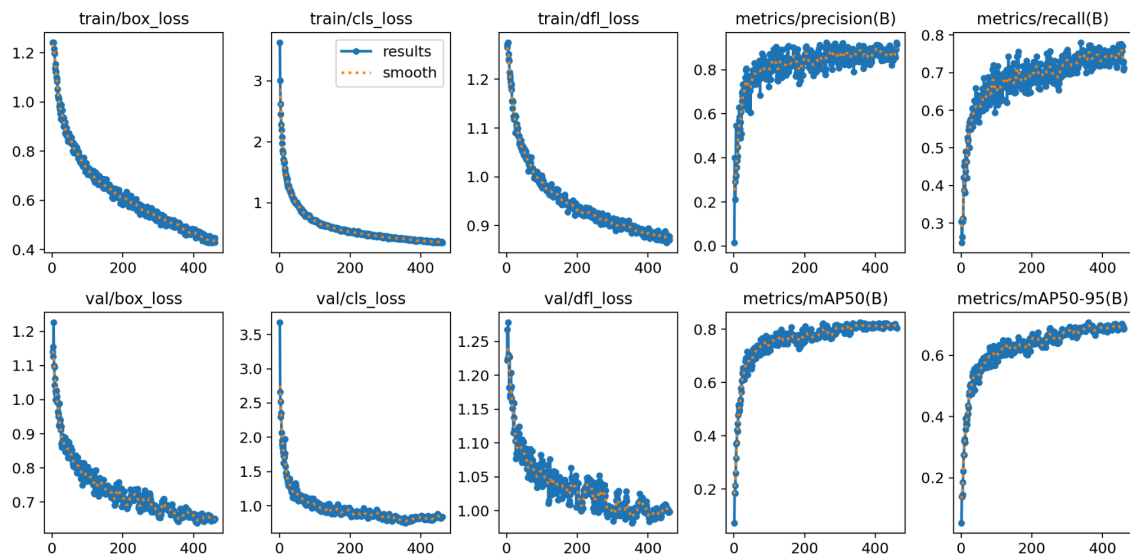


Figure 15 : Courbes de résultats du meilleur model

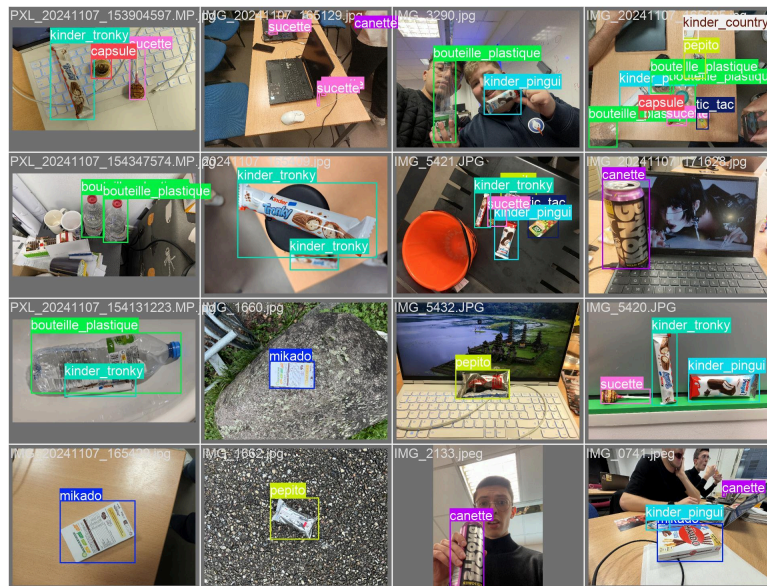
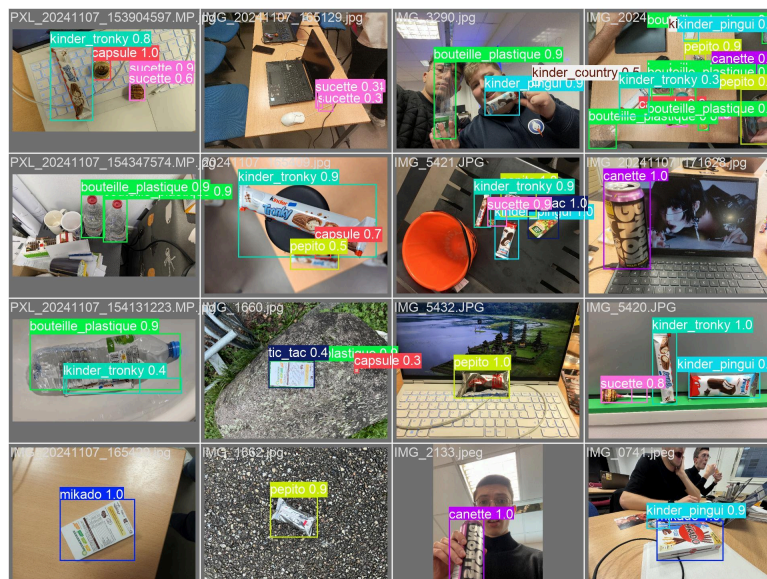


Figure 16 et 17 : Batch de validation (labels et pred)



<pre> epochs: 500 time: null patience: 100 batch: 8 imgsz: 640 save: true save_period: -1 cache: disk device: cuda:0 workers: 8 project: ./results name: train13 exist_ok: false pretrained: true optimizer: AdamW verbose: true seed: 42 deterministic: true single_cls: false rect: false cos_lr: false close_mosaic: 0 resume: false amp: true fraction: 1.0 workspace: null nms: false lr0: 0.001 lrf: 0.01 momentum: 0.937 weight_decay: 0.0005 warmup_epochs: 3.0 warmup_momentum: 0.8 warmup_bias_lr: 0.1 box: 7.5 cls: 0.5 dfl: 1.5 pose: 12.0 kobj: 1.0 nbs: 64 hsv_h: 0.015 hsv_s: 0.7 hsv_v: 0.4 degrees: 0.0 translate: 0.1 scale: 0.5 shear: 0.0 perspective: 0.0 flipud: 0.0 fliplr: 0.5 </pre>	<pre> profile: false freeze: null multi_scale: false overlap_mask: true mask_ratio: 4 dropout: 0.0 val: true split: val save_json: false save_hybrid: false conf: null iou: 0.7 max_det: 300 half: false dnn: false plots: true source: null vid_stride: 1 stream_buffer: false visualize: false augment: true agnostic_nms: false classes: null retina_masks: false embed: null show: false save_frames: false save_txt: false save_conf: false save_crop: false show_labels: true show_conf: true show_boxes: true line_width: null format: torchscript keras: false optimize: false int8: false dynamic: false simplify: true opset: null bgr: 0.0 mosaic: 1.0 mixup: 0.0 copy_paste: 0.0 copy_paste_mode: flip auto_augment: randaugment erasing: 0.4 crop_fraction: 1.0 cfg: null </pre>
---	---

Figure 18 : Liste complète des hyperparamètres (args.yaml)

Et s'il fallait plus grand ?

Finalement, nous n'avons pas essayé de modifier un des plus gros "paramètres" dans nos entraînements : la taille du modèle. Le model YOLO existe en plusieurs tailles. Par soucis de performance et de rapidité, nous avons préféré utiliser la version nano, la plus petite. Et si l'on essayait une version plus grande ? Les RTX 4090 ont encore été mises à profit et le résultat est un peu mieux grâce au model en taille medium (**YOLO11m**).

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Figure 19 : Les différentes tailles de model YOLO v11 (github.com)

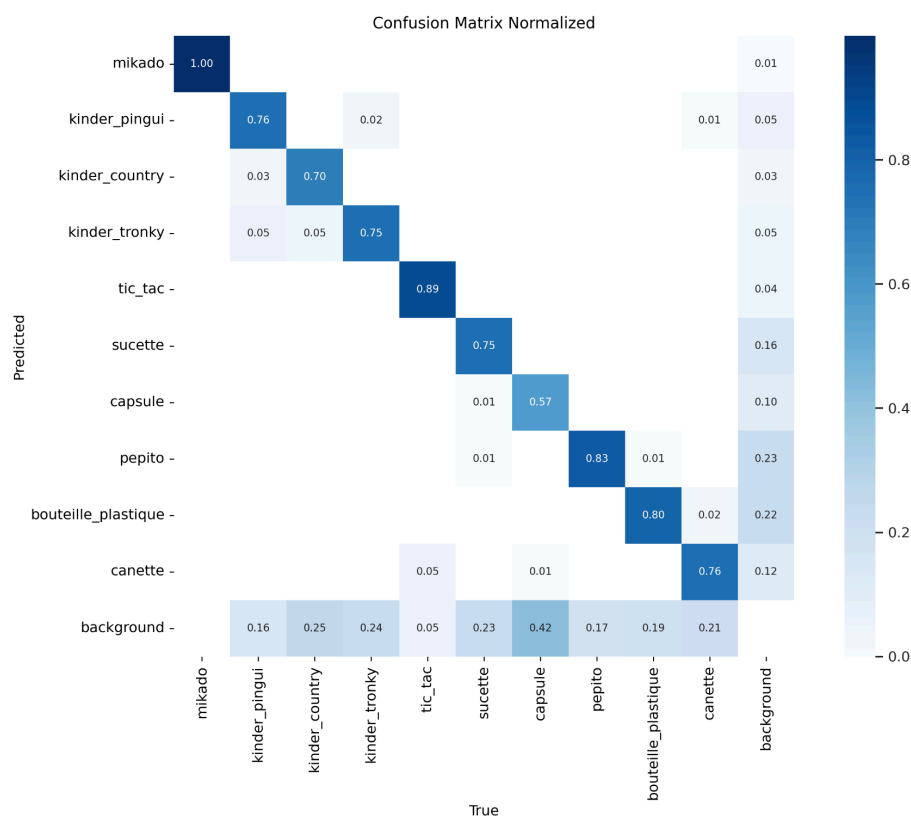


Figure 20 : Matrice de confusion du meilleur model avec YOLO11m

Destination finale : la taille large (**YOLO11l**). Est-ce mieux ? Malheureusement non mais au moins nous avons essayé.

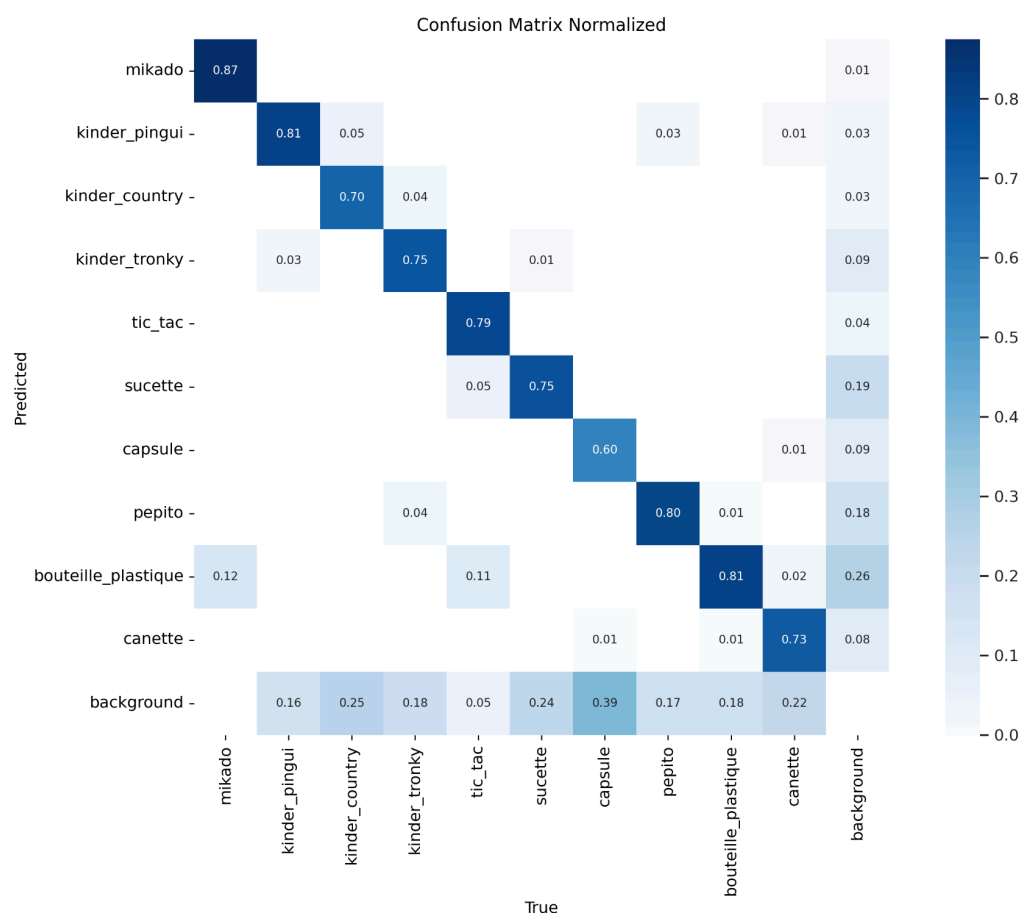


Figure 21 : Matrice de confusion de l'entraînement avec YOLO11l

Le meilleur modèle est donc celui que l'on a entraîné avec YOLO11m en gardant la même liste d'hyperparamètres par rapport à avant.

Lien vers notre meilleur model final : [Picsellia](#)

Pistes d'améliorations

Afin d'avoir un meilleur model, il faudrait déjà avoir moins de limitations de temps et plus de moyen. Il ne fait aucun doute qu'avoir une très bonne carte NVIDIA aide à expérimenter plus rapidement. Il serait peut-être judicieux de jouer sur les poids des classes (même si l'ensemble paraît assez homogène, ce que nous n'avons pas pensé à faire). La plus grande amélioration à faire est le problème des "faux positifs" et des "invisibles". Encore trop d'objets sont épargnés et passent à la trappe de notre model. (voir la ligne background dans chaque matrice de confusion). L'utilisation d'une meilleure version de YOLO 11 pourrait aussi fortement altérer la performance du modèle. Pour plus de facilité d'expérimentation, nous avons préféré YOLO 11 en taille nano puis finalement essayé des plus grandes versions. À voir avec une taille encore grande comme xl et avec un batch_size plus grand. Outre cela, la learning rate est bonne et les autres hyperparamètres principaux sont assez optimisés.

Notre pipeline de training

Pour finir ce rapport, un petit aperçu sur la pipeline de training du projet

